

Taller: Control de Calidad de Software

Unidad 1 - Testing Ágil - Tipos de Testing





- Testing Ágil
- Proceso de Prueba: Actividades
- Niveles de Prueba
- Tipos de Prueba
- Roles afectados a Niveles y tipos de prueba
- Pruebas Estáticas:
 - Concepto
 - Objetos de prueba
 - Roles

1990

Cliente servidor

Monolíticos

3 Capas

Desarrollo orientado a obj.

Tecnologías emergentes



2024

Microservicios

Contenedores

Computación en la nube

APIs y microservicios rest

Big Data

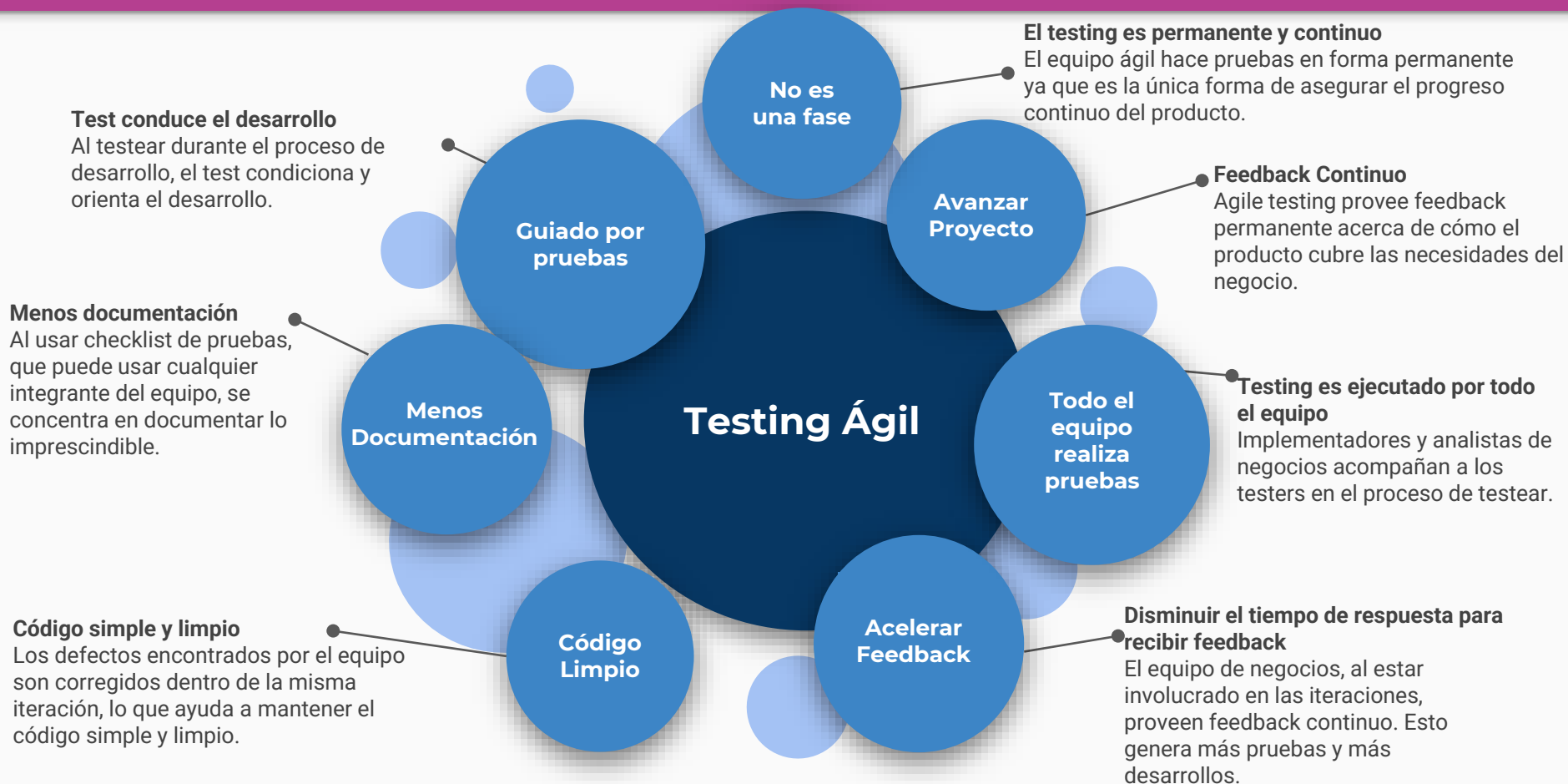
Seguridad centrada en la
arquitectura

Es la actividad de testing que sigue las reglas y principios del desarrollo de software ágil.

Debe comenzar al inicio del proyecto con pruebas tempranas del software que se está desarrollando.

No es secuencial, es continuo.

Durante todo el proceso de desarrollo se aconsejan diferentes formas de testing del producto que se está desarrollando.



- **Test-Driven Development (TDD)**

Es una técnica de desarrollo que consiste en escribir primero las pruebas (generalmente unitarias), después escribir el código fuente que pase la prueba satisfactoriamente y, por último, refactorizar el código escrito.

- **Acceptance Test-Driven Development (ATDD)**

Es una dimensión del TDD aplicada al nivel de gestión de requerimientos de software, en el cual las pruebas escritas son a nivel de cliente, es decir, lo equivalente a una prueba de aceptación o test funcional.

- **Behaviour Driven Development (BDD)**

Es un enfoque donde primero se desarrollan las pruebas de aceptación escribiendo criterios de aceptación en las historias de usuario. Luego se ejecuta el desarrollo aplicando TDD hasta que la prueba es exitosa.

- **Testing exploratorio**

Enfoque en el cual el aprendizaje de la funcionalidad, diseño de pruebas y ejecución de pruebas ocurren simultáneamente, en contraposición con el enfoque convencional en el cual primero se documenta la funcionalidad o requisito, luego se diseña el caso de prueba y luego se ejecuta de acuerdo a guiones preestablecidos.

- **Automatización de pruebas de regresión**

Tanto la integración continua como la refactorización son prácticas necesarias para poder implementar una metodología ágil de desarrollo de software.

- **Automatización de pruebas unitarias**

Consiste en usar un marco de trabajo o framework para ejecutar tests unitarios, en lugar de ejecutar estos manualmente una y otra vez cada vez que se modifica el código.

Proceso de pruebas - Actividades

- **Planificación:** se definen los objetivos y el enfoque para cumplirlos.
- **Seguimiento y Control:** comparación continua del avance real vs plan; tomar medidas necesarias para cumplir el objetivo. Informes de avance.
- **Análisis:** se identifican las características del objeto de prueba: “que probar”.
- **Diseño:** se definen casos de prueba y sets: “como probar”
- **Implementación:** se desarrollan procedimientos, se organiza un calendario de ejecución, se construyen entornos y se preparan datos de prueba.
- **Ejecución:** se ejecutan las suites de prueba siguiendo el calendario.
- **Compleción:** se recopilan datos de las actividades completadas: informes de defectos, analizar lecciones aprendidas e identificar mejoras.



Niveles de Prueba

- Prueba de Componente
- Prueba de Integración
- Prueba de Sistema
- Prueba de Aceptación

- Objetivos
- Bases de Prueba
- Objeto de Prueba
- Defectos y Fallos
- Enfoques y Responsabilidades



Entorno de prueba adecuado



- Unitarias
- Automatizada
- Aislada
- Posiblemente requiera simulaciones
- Permite la corrección rápida de fallos. Suelen no reportarse, excepto para análisis de causa raíz.
- Pruebas escritas por desarrolladores, o roles que conozcan el código de la aplicación.



Bases de las pruebas:

- Requerimientos
- Diseño detallado
- Código

Objeto de pruebas:

- Componentes
- Programas
- Conversión de datos/migración de programas
- Módulos de bases de datos



- Basada en interacciones entre componentes
- Automatizar para garantizar estabilidad
- Integración incremental para detectar y aislar defectos en forma temprana.
- Dos niveles:
 - Entre componentes integrados (automatizar)
 - Integraciones con aplicaciones externas (pruebas manuales)

Bases de las pruebas:

- Diseño de sistema
- Arquitectura
- Flujos de trabajo
- Casos de uso

Objeto de pruebas:

- Subsistemas
- Implem. de base de datos
- Infraestructura
- Interfaces
- Conf. de sistemas y de datos





- Pruebas E2E (end-to-end) funcionales y no funcionales
- Verifica requisitos y/o estándares legales o regulatorios
- Entorno de prueba = entorno productivo (Idealmente)

Bases de las pruebas:

- Especificación de req. de Soft. y de Sistema
- Casos de uso
- Especificación Funcional
- Reporte de análisis de riesgo.

Objeto de pruebas:

- Manuales de operación, de usuario y de sistema
- Datos de configuración y conf. del sistema



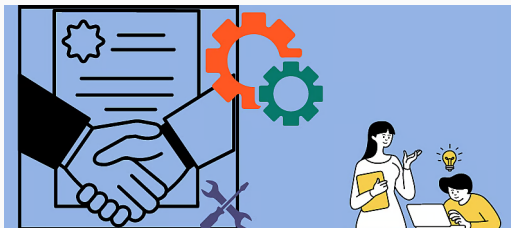
- Se centra en el comportamiento de todo el sistema.
- Produce información para decidir si el sistema puede o no desplegarse.
- Tipos:
 - Del usuario (UAT)
 - operativa (recuperación de desastres, admin de usuarios, mantenimiento, migraciones, vulnerabilidades, otras)
 - contractual o de regulación
 - Alfa (organización que la desarrolla) y beta (potenciales clientes)

Bases de las pruebas:

- Req. del Usuario
- Req. del sistema
- Casos de uso
- Proceso de Negocio
- Reporte de análisis de riesgo.

Objeto de pruebas:

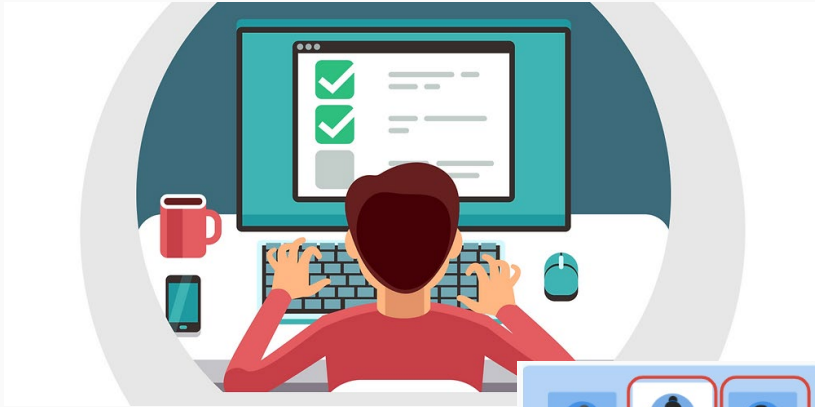
- Procesos de negocio integrados completamente
- Procesos de mantenimiento y operación
- Procedimientos de usuario
- Reportes
- Datos de Configuración



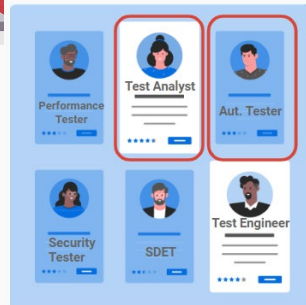
Tipos de Prueba

- Funcional
- No Funcional
- De Caja Blanca
- Asociada al cambio

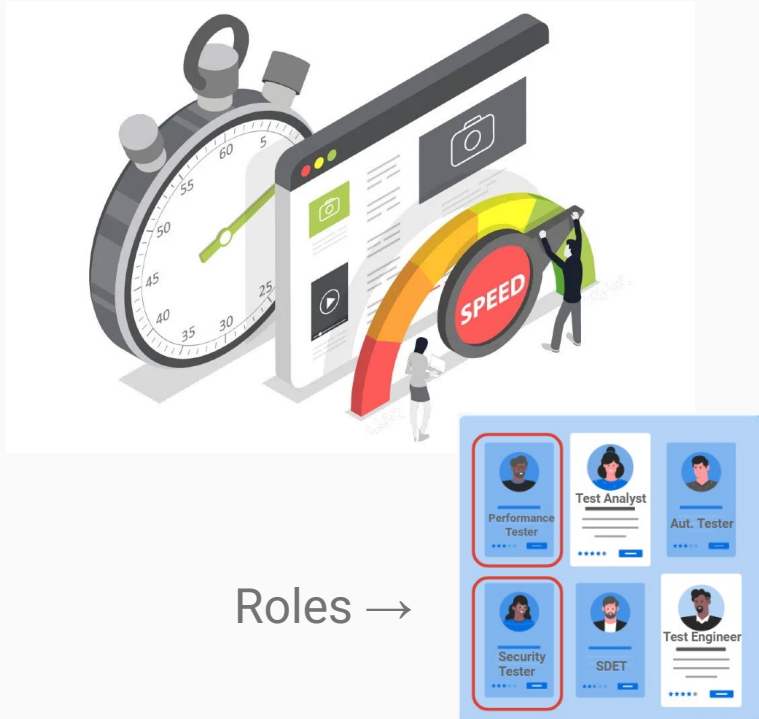
Basados en Objetos de Prueba
específicos



Roles →

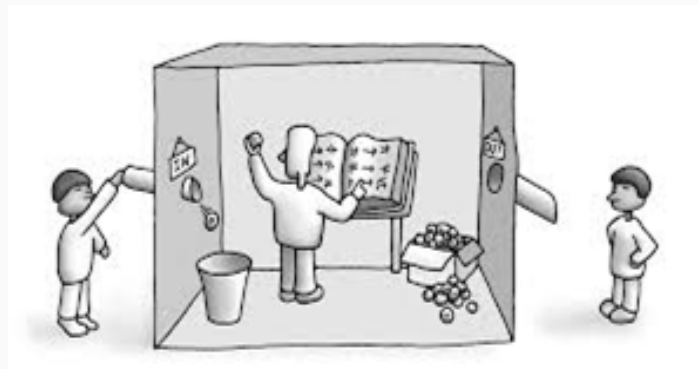


- Evalúan funciones del software vs. especificaciones funcionales, épicas o HU (Historia de Usuario)
- “QUÉ” hace el sistema
- Se realiza en todos los niveles de prueba, pero con diferentes enfoques.
- Se usan técnicas de caja negra para condiciones de prueba.
- Casos de prueba por funcionalidad.
- Intensidad de las pruebas está dada por la **cobertura**.
- Objeto de pruebas: software funcionando en algún entorno

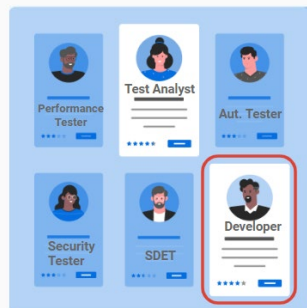


Roles →

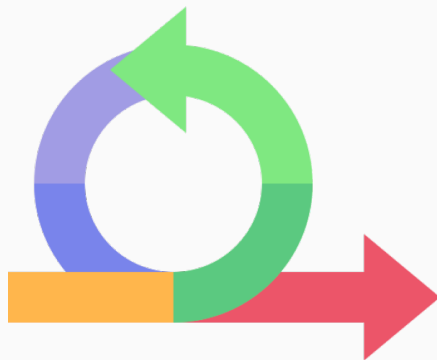
- Evalúan características del sistema como:
 - usabilidad
 - performance (carga, concurrencia, tiempos de respuesta, etc.)
 - seguridad
- Evalúan “CÓMO DE BIEN” se comporta el sistema.
- Se puede realizar en todos los niveles de prueba.
- Debe hacerse tan pronto como sea posible, no debe esperarse al final de desarrollo.
- Intensidad de las pruebas está dada por la **cobertura x items no funcionales**.



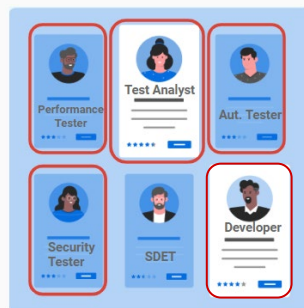
Roles →



- Basadas en la estructura interna o en la implementación.
 - código
 - arquitectura
 - flujos de datos o de trabajo
- Si se realiza a nivel de componentes, el objeto de prueba es el **código**. Dos niveles:
 - Cobertura de código
 - Cobertura de sentencia
- Si se realiza a nivel de integración, el objeto de pruebas es la **arquitectura**.



Roles →



- Prueba de **Confirmación**, para garantizar el cambio realizado.
- Prueba de **Regresión**, para garantizar que otras funcionalidades no se vieron afectadas.
- En todos los niveles de pruebas
- Especialmente en ciclos iterativos



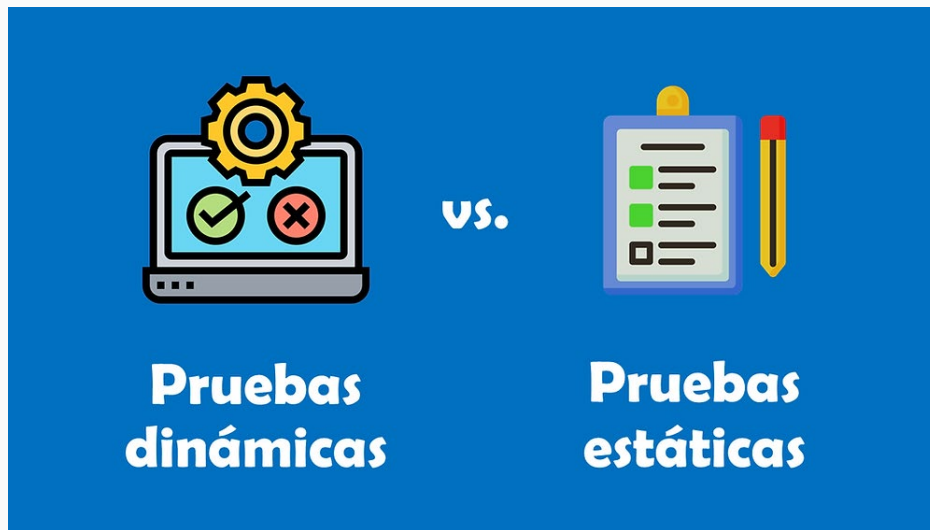
Pruebas Estáticas

- Se realizan **sin ejecutar código** de la aplicación.
- Productos de trabajo evaluados:
 - Especificaciones
 - Epicas, HU, Criterios de aceptación
 - Especificaciones de arquitectura y diseño
 - Código (pero sin ejecutar)
 - Productos de prueba: planes de prueba, casos de prueba, procedim de prueba, Scripts de prueba automatizados.
 - Guías o manuales de usuario

- Proceso formal y estructurado:
 - Listas de verificación
 - Roles que intervienen: Moderador, Productor, Revisores, Escriba
- Foco u objetivo: identificar defectos
- Genera datos o información que retroalimenta el proceso de calidad.

¿Cuál es más importante?

¿Qué priorizamos?



¿Qué roles de Testing deben intervenir?

¿Preguntas?



GRACIAS

